SBA301 – SLOT 31

CHAPTER 18 – HIBERNATE APPLICATION DEVELOPMENT & CRUD OPERATIONS

PART B: JAVAFX • FXML • CONTROLLER • CRUD UI INTEGRATION

CORE: Tư duy trung tâm của Slot 31: JavaFX/FXML là presentation layer, không phải persistence layer. FXML mô tả scene graph/UI structure; Controller điều phối interaction; Service là boundary để Controller gọi nghiệp vụ. Một StudentController tốt không mở Hibernate Session, không viết HQL, không chứa transaction logic và không nhồi toàn bộ behavior vào event handler.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	31
Chủ đề	Chapter 18 Part B – JavaFX / FXML / Controller
CLO chính	CLO5 – Integrate JavaFX presentation layer with layered Hibernate CRUD application
Ngữ liệu xuyên suốt	Student Management GUI – Chapter 18 + Lab 07
Course baseline	JDK 21 • JavaFX course baseline • Maven • FXML • Scene Builder • Hibernate Service layer
Thời lượng gợi ý	135 phút + implementation/self-study
Learning flow	Prepare → Understand Scene Graph → Design FXML → Wire Controller → Bind Table → CRUD Events → Break → Diagnose → Verify

NOTE: Scope lock: Slot 31 tập trung JavaFX, FXML, FXMLLoader, Controller, TableView/Form CRUD integration và module configuration. Không đưa Hibernate code vào Controller. Login page của Lab 07 chỉ được nhắc như bridge; không phải trọng tâm chính.

CÁCH SỬ DỤNG GIÁO TRÌNH

Theo template Slot 22: Phần IV là knowledge base lý thuyết; Phần VI là workbook triển khai từ project/module/FXML tới Controller và CRUD UI. Sinh viên phải kiểm ba loại evidence: UI state, Controller/service call trace và database result.

Pha	Sinh viên làm gì?	Evidence
Prepare	Chạy Slot 30 service CRUD không cần UI.	Service baseline
Understand	Scene graph, FXML, Controller lifecycle.	Concept map
Design	Tạo student-view.fxml/Scene Builder.	FXML
Wire	fx:controller, fx:id, @FXML, module opens.	Load evidence
Bind	TableView/TableColumn/form selection.	UI evidence
CRUD	Create/Update/Delete/Refresh qua Service.	UI + DB evidence
Break	Sai fx:id, controller, module, conversion, validation.	Failure evidence
Diagnose	FXMLLoadException, stack trace, debugger.	Debug note
Verify	CRUD matrix + controller/service separation.	Integration gate

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

0.1. Kiến thức cần ôn lại

- Slot 30: StudentDAO → Repository → Service và full CRUD contract.
- Java OOP: fields, methods, constructors, event handling.
- Java collections và ObservableList awareness.
- Basic XML syntax: element, attribute, hierarchy.
- Java module system awareness: requires/exports/opens.
- UI concepts: form, table, selection, validation, feedback.

0.2. Baseline cần chạy trước buổi học

Check	PASS condition
StudentService	Create/read/update/delete chạy bằng Main
LibraryDB	Có dữ liệu Student test
Maven	Build sạch
JavaFX	Project/plugin/dependencies resolve
Scene Builder	Mở được FXML
Git	Checkpoint sạch

Baseline commands

```
java -version
mvn -v
mvn -q -DskipTests compile
mvn test

# JavaFX module/project sau khi cấu hình:
mvn clean javafx:run
```

0.3. Pre-class 3–2–1

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi cần chứng minh
Service có CRUD; FXML mô tả UI; Controller xử lý event.	FXMLLoader inject @FXML thế nào? Khi nào initialize() chạy?	Từ click nút Save tới DB INSERT, những lớp nào được gọi?

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1.1. Progression từ Chapter 18 source

Nguồn Chapter 18/Lab 07	Slot 31 triển khai
Create JavaFX/Maven project	Project + dependencies
Build architecture	Presentation package + service reference
Edit module-info.java	requires javafx.controls/fxml; opens controller
Add reference another project	Service-layer dependency
Open student-view.fxml in SceneBuilder	FXML design
Edit StudentController	@FXML injection + event handlers
Show all/get ID on first cell	TableView + selection model
CRUD for Students	Create/update/delete/refresh
Run Project	Integration verification

1.2. Learning objectives

1. Giải thích JavaFX Application/Stage/Scene/Node/scene graph.
2. Giải thích FXML là XML-based declarative UI và vai trò của FXMLLoader.
3. Phân biệt FXML view, Controller, Service và persistence layers.
4. Cấu hình Maven/module-info cho javafx.controls và javafx.fxml.
5. Sử dụng `fx:controller`, `fx:id`, `@FXML`, event handler và `initialize()` đúng.
6. Thiết kế Student form + TableView bằng FXML/Scene Builder.
7. Bind TableColumn với Student properties và TableView với ObservableList.
8. Đồng bộ row selection → form fields.
9. Triển khai Create/Update/Delete/Refresh qua `IStudentService`.
10. Thiết kế validation, alert/error feedback và reset form.
11. Chẩn đoán FXMLLoadException, null @FXML field, module reflection errors.
12. Nhận biết JavaFX Application Thread và tránh chạy DB operation dài trên UI thread trong production.

1.3. Mental model

```
FXML file
  |
  | parsed by
  v
FXMLLoader
  |
  +-- creates Scene Graph (VBox, TableView, TextField, Button...)
  |
  +-- creates/connects StudentController
  |       |
  |       +-- injects @FXML fields by fx:id
  |       +-- calls initialize()
  |
```

```
  v
User Interaction
  |
  +-- onAction="#handleSave"
  v
StudentController
  |
  v
IStudentService
  |
  v
Repository -> DAO -> Hibernate -> SQL Server
```

1.4. Separation rule

Class/File	Được phép biết	Không nên chứa
student-view.fxml	UI nodes/layout/event names	Hibernate/SQL/business logic
StudentController	UI state + Service contract	Session/HQL/DB credentials
StudentService	business/use-case	JavaFX Node/FXML
Repository/DAO	persistence	Button/TableView

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

2.1. JavaFX Application lifecycle

`Application.start(Stage)` là entry vào UI. Stage là top-level window; Scene chứa scene graph.

2.2. Scene graph

UI là cây Nodes. Parent/child hierarchy quyết định layout/rendering.

2.3. FXML

FXML là XML-based markup để xây Java object graph; rất phù hợp khai báo scene graph.

2.4. FXMLLoader

FXMLLoader đọc FXML, tạo Nodes, controller và injection/event wiring.

2.5. fx:controller

Khai báo controller class được gắn với FXML document.

2.6. fx:id

Identifier cho object trong FXML để Controller có thể được inject/tham chiếu.

2.7. @FXML

Cho phép FXMLLoader truy cập fields/methods không public; trong named module package phải được reflectively accessible.

2.8. initialize()

Được gọi sau khi FXML fields đã được inject; phù hợp để cấu hình TableColumn/listeners và initial refresh.

2.9. onAction handler

FXML có thể gọi controller method như `onAction="#handleSave"`.

2.10. TableView

Hiển thị collection rows; dữ liệu thường được quản lý qua ObservableList.

2.11. TableColumn

Cần cell value mapping tới Student properties; có thể dùng PropertyValueFactory hoặc explicit cellValueFactory.

2.12. SelectionModel

Theo dõi selected row; listener dùng để đưa data lên form.

2.13. ObservableList

Collection observable giúp UI phản ứng khi danh sách thay đổi; thường refresh từ Service sau mutation.

2.14. Controller state

selectedStudentId hoặc selectedStudent giúp phân biệt Create vs Update.

2.15. Form validation

Parse/type/range/required checks xảy ra trước Service call; business validation vẫn thuộc Service.

2.16. Alert/feedback

Controller chuyển success/failure thành user feedback; không show raw stack trace.

2.17. Reset form

Clear fields + clear selection + reset selected ID sau create/delete.

2.18. Refresh strategy

Sau CRUD, reload list từ Service là course-safe; production có thể update ObservableList có kiểm soát.

2.19. JavaFX Application Thread

UI modifications phải chạy trên FX Application Thread. DB operation dài sẽ freeze UI nếu chạy đồng bộ.

2.20. Background Task awareness

Production direction: Task/Service/CompletableFuture + UI update on FX thread cho blocking work.

2.21. JPMS reflection

Nếu modular app, controller package cần `opens ... to javafx.fxml`; chỉ `exports` không luôn đủ cho private @FXML injection.

2.22. FXML/Controller wiring matrix

FXML construct	Controller counterpart	Failure if wrong
fx:controller	StudentController class	controller load error
fx:id="txtEmail"	@FXML TextField txtEmail	field remains null/injection error
onAction="#handleSave"	@FXML void handleSave()	method resolution error
TableView fx:id	@FXML TableView<Student>	cannot bind/listen
module opens controller package	reflection access	InaccessibleObjectException

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

3.1. Student Management screen

UI component	Responsibility	Service interaction
TableView<Student>	List/selection	getAll()
Email TextField	input/display	create/update DTO/entity
First/Last Name	input/display	create/update
Marks TextField	numeric input	validation
Save/Create button	submit new	create()
Update button	modify selected	update()
Delete button	remove selected	delete()
Refresh button	reload	getAll()

3.2. Event-to-DB sequence

```
User clicks Save
  -> StudentController.handleSave()
  -> read + validate form
  -> StudentService.create(student)
```

```
-> Repository.save()
-> DAO.save()
-> Hibernate INSERT
-> return Student
-> Controller refreshTable()
-> TableView displays persisted data
```

3.3. Production transfer

Course pattern	Production direction
Controller creates/wires Service manually	DI/composition root/controller factory
DB call in event handler	background Task/Service
PropertyValueFactory string names	typed lambdas/properties where possible
Alert for every error	central UI error/validation patterns
Full table reload	incremental updates/paging
FXML + Controller	MVVM/MVP variants where complexity grows

IV. NỘI DUNG CHI TIẾT – KNOWLEDGE BASE JAVAFX / FXML / CONTROLLER

READING GUIDE: Phần IV là theory/reference. Phần VI mới là tutorial thao tác.

4.1. JavaFX architecture

- Toolkit gồm controls, graphics, FXML, properties/binding, concurrency và media/web modules tùy app.
- Slot 31 tập trung controls + fxml.

4.2. Application, Stage, Scene

- Application lifecycle tạo primary Stage.
- Scene chứa root Node và graph UI.

4.3. Node hierarchy

- Control/Layout đều là Nodes.
- Layout pane quản lý size/position children.

4.4. Declarative vs imperative UI

- FXML tách structural UI khỏi Java controller logic.
- Không phải toàn bộ UI logic đều cần FXML; nhưng source Chapter 18 dùng FXML/Scene Builder.

4.5. FXML object construction

- Element có thể đại diện class instance/property/static property.
- FXML hierarchy gần với scene graph hierarchy.

4.6. FXMLLoader lifecycle

- Read resource → instantiate hierarchy/controller → inject fields → call initialize.
- Resource path/controller errors thường phát hiện khi load.

4.7. fx:controller

- Gắn compiled Java controller với FXML document.
- Không vừa dùng fx:controller vừa setController tùy tiện nếu không hiểu loader semantics.

4.8. fx:id

- Tạo namespace ID và phục vụ injection/reference.
- fx:id khác CSS `id` dù có liên quan trong một số cases.

4.9. @FXML annotation

- Đánh dấu field/method để loader truy cập.
- Modular project cần package opened to javafx.fxml.

4.10. initialize()
- Chạy sau injection.
- Không dùng constructor để access @FXML fields vì lúc đó chưa inject.

4.11. Controller constructor
- Dùng cho non-FXML dependencies nếu custom controller factory/DI.
- Course đơn giản thường setter/static composition nhưng cần hiểu limitation.

4.12. Event handlers
- onAction maps UI events tới controller methods.
- Handler nên mỏng: validate → call service → update UI.

4.13. TableView model
- TableView quản lý items và selection.
- Không dùng TableView như database; DB source-of-truth nằm qua Service.

4.14. ObservableList
- UI-observable collection cho items.
- Sau service mutation có thể reload hoặc mutate list có kiểm soát.

4.15. TableColumn mapping
- PropertyValueFactory dùng property name convention/reflection.
- Typed callback giúp refactor-safe hơn.

4.16. Selection listener
- selectedItemProperty listener giúp đưa row data vào form.
- Phải handle null selection.

4.17. Create mode vs edit mode
- Controller cần explicit state hoặc selection để biết operation.
- Không suy ra mơ hồ chỉ từ field text.

4.18. Input parsing
- TextField chỉ chứa String; marks/id cần parse.
- NumberFormatException là presentation validation issue.

4.19. Validation layers
- UI: format/required/immediate feedback.
- Service: business invariants; DB: constraints.

4.20. Error presentation
- Catch expected application exceptions và show Alert/message.
- Unexpected technical errors nên log và show generic message.

4.21. Refresh patterns
- Course: call service.getAll() sau CRUD.
- Production: update observable model/incremental/paging.

4.22. Focus/selection reset
- Clear form, selection và button mode sau operation.
- State consistency quan trọng để tránh update nhầm record.

4.23. Controller coupling
- Controller phụ thuộc Service interface, không DAO/Hibernate.
- Giảm test cost và giữ UI replaceable.

4.24. Controller testability
- Extract validation/formatting helpers; inject fake/mock service.
- FXML integration test riêng cho wiring.

4.25. JavaFX Application Thread
- Event handlers chạy trên FX Application Thread.
- Blocking DB call dài gây UI freeze.

4.26. Task/Service concurrency
- JavaFX `Task`/`Service` hỗ trợ background work và lifecycle events.
- Không update controls trực tiếp từ arbitrary background thread.

4.27. module-info.java
- requires javafx.controls; requires javafx.fxml.
- Controller package used by FXML cần opens to javafx.fxml.

4.28. exports vs opens
- exports cho compile-time/public access.
- opens cho deep reflection/runtime access; FXML loader cần reflection.

4.29. Maven JavaFX
- OpenJFX docs hỗ trợ Maven dependencies và javafx-maven-plugin.
- FXML cần javafx.fxml dependency/module.

4.30. Scene Builder role
- Visual designer tạo/edit FXML.
- Generated FXML vẫn phải code review và hiểu hierarchy/fx:id/controller.

4.31. CSS awareness
- Style nên tách CSS khi UI lớn.
- Không hard-code mọi style inline FXML.

4.32. Navigation
- Scene switching/load new FXML cần lifecycle/dependency management.
- Không tạo Service/SessionFactory mới mỗi navigation.

4.33. Resource loading
- FXML/CSS/image resources nên nằm resources classpath.
- Sai path là lỗi runtime phổ biến.

4.34. Controller factory/DI
- FXMLLoader có controllerFactory để tích hợp dependency injection.
- Production direction hữu ích khi Service không phải singleton/static global.

4.35. Avoid static global service
- Dễ làm demo nhưng gây hidden dependency/test khó.
- Composition root/controller factory tốt hơn khi app lớn.

4.36. Table performance

- Large datasets không nên load hết vào TableView.
- Paging/filter/search nên đặt Service/Repository.

4.37. JavaFX properties

- Property/Binding giúp synchronize UI state.
- Course có thể dùng POJO getters; advanced direction là JavaFX properties/view model.

4.38. FXML load failures

- fx:id mismatch, handler missing, bad controller, package inaccessible, malformed XML/resource not found.
- Đọc nested cause, line number và module error.

4.39. Security/validation awareness

- Desktop UI không thay service validation.
- Không hard-code DB credentials trong UI project.

4.40. Current OpenJFX direction

- Official OpenJFX docs tiếp tục hướng dẫn Maven/Gradle modular applications; FXML projects require `javafx.fxml`.
- FXML controller/custom Node packages trong named modules phải được opened reflectively to javafx.fxml.

V. KINH NGHIỆM THỰC TIỄN, TIPS

Lỗi	Triệu chứng	Nguyên nhân thường gặp	Chẩn đoán
FXML not found	Load exception	resource path sai	getResource()
Controller null field	NPE in initialize/event	fx:id mismatch/@FXML/module	FXML + module
Handler not found	FXMLLoadException	onAction method sai	method signature/name
InaccessibleObjectException	FXML load fail	package not opened	module-info
Table empty	UI chạy nhưng không data	items/refresh/service	debug list size
Update wrong row	record khác bị sửa	selection state stale	selected ID
Marks parse error	NumberFormatException	raw text invalid	UI validation
UI freeze	click xong treo	DB call on FX thread	thread dump/timing

5.1. Debugging decision tree

```
JavaFX screen fails
|
+-- FXML doesn't load?
|   +-- resource path?
|   +-- malformed XML?
|   +-- fx:controller class?
|   +-- module opens?
|
+-- @FXML field null?
|   +-- fx:id exact match?
|   +-- @FXML present/private?
|   +-- initialize timing?
|
+-- Button click does nothing?
|   +-- onAction method name?
|   +-- handler signature?
|
+-- CRUD not reflected?
|   +-- service call happened?
|   +-- DB changed?
|   +-- refreshTable() called?
|
+-- UI freezes?
    +-- blocking DB work on FX Application Thread?
```

- Đừng access @FXML fields trong Controller constructor.

- `fx:id` và field name phải khớp nếu dùng injection.
- Controller chỉ nên gọi Service.
- Sau CRUD course-safe: refresh list rồi clear form/selection.
- Phân biệt UI validation và Service business validation.
- Không dùng Alert để che technical bug; log stack trace ở dev.
- Scene Builder tạo FXML nhưng vẫn phải đọc được FXML bằng tay.
- Không mở/đóng Hibernate Session trong JavaFX event handler.

VI. HƯỚNG DẪN STEP BY STEP THỰC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

WORKBOOK RULE: Mỗi bước: Mục tiêu → thao tác → code/config → run → verification → PASS/FAIL.

6.1. Step 1 – Tạo JavaFX Maven project / module presentation

```
student-ui/
├─ pom.xml
└─ src/main/
   ├─ java/
   │  ├─ module-info.java
   │  └─ fu/edu/sba301/ui/
   │     ├─ StudentApplication.java
   │     └─ controller/
   │        └─ StudentController.java
   └─ resources/
      └─ fu/edu/sba301/ui/
         ├─ student-view.fxml
         └─ student.css
```

PASS: compile before adding Hibernate/UI integration.

6.2. Step 2 – JavaFX dependencies

pom.xml skeleton

```
<dependencies>
  <dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>${javafx.version}</version>
  </dependency>
  <dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-fxml</artifactId>
    <version>${javafx.version}</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-maven-plugin</artifactId>
      <version>${javafx.maven.plugin.version}</version>
      <configuration>
        <mainClass>
          fu.edu.sba301.ui/fu.edu.sba301.ui.StudentApplication
        </mainClass>
      </configuration>
    </plugin>
```

```
  </plugins>
</build>
```

VERSION: Dùng version đã khóa trong course/project. OpenJFX official docs xác nhận FXML cần module/dependency `javafx.fxml`.

6.3. Step 3 – module-info.java

```
module fu.edu.sba301.ui {
    requires javafx.controls;
    requires javafx.fxml;

    // nếu service/domain là module khác:
    requires fu.edu.sba301.core;

    opens fu.edu.sba301.ui.controller to javafx.fxml;
    exports fu.edu.sba301.ui;
}
```

PASS: FXMLLoader có quyền reflection vào Controller package.

COURSE: Nếu project không dùng JPMS thì file module-info có thể không áp dụng; nhưng Chapter/Lab source có module-info nên Slot 31 giữ nội dung này.

6.4. Step 4 – StudentApplication load FXML

```
public class StudentApplication extends Application {

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader loader = new FXMLLoader(
            StudentApplication.class.getResource(
                "/fu/edu/sba301/ui/student-view.fxml"
            )
        );

        Scene scene = new Scene(loader.load());
        stage.setTitle("Student Management");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Run

```
mvn clean javafx:run
```

PASS: empty/basic screen loads without exception.

6.5. Step 5 – Create `student-view.fxml` root layout

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<BorderPane
    xmlns="http://javafx.com/javafx"
    xmlns:fx="http://javafx.com/fxml"
    fx:controller="fu.edu.sba301.ui.controller.StudentController">

    <top>
```

```
        <Label text="Student Management"
               style="-fx-font-size: 20px; -fx-font-weight: bold;"/>
    </top>

    <center>
        <VBox spacing="10">
            <!-- form + table added in next steps -->
        </VBox>
    </center>
</BorderPane>
```

6.6. Step 6 – Add input form

```
<GridPane hgap="10" vgap="8">
    <Label text="Email" GridPane.rowIndex="0"/>
    <TextField fx:id="txtEmail"
               GridPane.columnIndex="1"
               GridPane.rowIndex="0"/>

    <Label text="First name" GridPane.rowIndex="1"/>
    <TextField fx:id="txtFirstName"
               GridPane.columnIndex="1"
               GridPane.rowIndex="1"/>

    <Label text="Last name" GridPane.rowIndex="2"/>
    <TextField fx:id="txtLastName"
               GridPane.columnIndex="1"
               GridPane.rowIndex="2"/>

    <Label text="Marks" GridPane.rowIndex="3"/>
    <TextField fx:id="txtMarks"
               GridPane.columnIndex="1"
               GridPane.rowIndex="3"/>
</GridPane>
```

Verification: Scene Builder preview hoặc runtime layout.

6.7. Step 7 – Add action buttons

```
<HBox spacing="8">
    <Button text="Create" onAction="#handleCreate"/>
    <Button text="Update" onAction="#handleUpdate"/>
    <Button text="Delete" onAction="#handleDelete"/>
    <Button text="Clear" onAction="#handleClear"/>
    <Button text="Refresh" onAction="#handleRefresh"/>
</HBox>
```

PASS: FXMLLoader resolve được handler names khi Controller hoàn thiện.

6.8. Step 8 – Add TableView

```
<TableView fx:id="tblStudents" prefHeight="360">
    <columns>
        <TableColumn fx:id="colId" text="ID" prefWidth="70"/>
        <TableColumn fx:id="colEmail" text="Email" prefWidth="180"/>
        <TableColumn fx:id="colFirstName" text="First Name" prefWidth="130"/>
        <TableColumn fx:id="colLastName" text="Last Name" prefWidth="130"/>
        <TableColumn fx:id="colMarks" text="Marks" prefWidth="90"/>
    </columns>
</TableView>
```

6.9. Step 9 – Controller @FXML fields

```
public class StudentController {

    @FXML private TextField txtEmail;
    @FXML private TextField txtFirstName;
    @FXML private TextField txtLastName;
    @FXML private TextField txtMarks;

    @FXML private TableView<Student> tblStudents;
    @FXML private TableColumn<Student, Long> colId;
    @FXML private TableColumn<Student, String> colEmail;
    @FXML private TableColumn<Student, String> colFirstName;
    @FXML private TableColumn<Student, String> colLastName;
    @FXML private TableColumn<Student, Double> colMarks;

    private final ObservableList<Student> data =
        FXCollections.observableArrayList();

    private IStudentService studentService;
    private Long selectedStudentId;
}
```

6.10. Step 10 – Inject/wire Service

Course-safe simple option: composition after FXMLLoader load hoặc Controller factory. Tránh tạo Hibernate DAO ngay trong Controller.

```
// In Application/composition root:
FXMLLoader loader = new FXMLLoader(...);
Parent root = loader.load();

StudentController controller = loader.getController();
controller.setStudentService(studentService);
```

Controller

```
public void setStudentService(IStudentService service) {
    this.studentService = service;
    refreshTable();
}
```

ARCHITECTURE: Không gọi `new StudentDAO()` trong StudentController.

6.11. Step 11 – initialize TableColumn

```
@FXML
private void initialize() {
    colId.setCellValueFactory(
        new PropertyValueFactory<>("id"));
    colEmail.setCellValueFactory(
        new PropertyValueFactory<>("email"));
    colFirstName.setCellValueFactory(
        new PropertyValueFactory<>("firstName"));
    colLastName.setCellValueFactory(
        new PropertyValueFactory<>("lastName"));
    colMarks.setCellValueFactory(
        new PropertyValueFactory<>("marks"));

    tblStudents.setItems(data);

    tblStudents.getSelectionModel()
        .selectedItemProperty()
        .addListener((obs, oldValue, selected) ->
```

```
            populateForm(selected));
}
```

LIFECYCLE: Không gọi service trong initialize nếu dependency chưa được set; dùng setter rồi refresh hoặc custom controllerFactory.

6.12. Step 12 – populate form from selected row

```
private void populateForm(Student s) {
    if (s == null) {
        return;
    }

    selectedStudentId = s.getId();
    txtEmail.setText(s.getEmail());
    txtFirstName.setText(s.getFirstName());
    txtLastName.setText(s.getLastName());
    txtMarks.setText(Double.toString(s.getMarks()));
}
```

PASS: click row → form displays đúng Student và selected ID.

6.13. Step 13 – Refresh TableView

```
private void refreshTable() {
    if (studentService == null) {
        return;
    }

    data.setAll(studentService.getAll());
}
```

Verification: DB baseline records xuất hiện.

6.14. Step 14 – Parse & validate form

```
private Student readForm() {
    String email = txtEmail.getText().trim();
    String first = txtFirstName.getText().trim();
    String last = txtLastName.getText().trim();

    if (email.isBlank() || first.isBlank() || last.isBlank()) {
        throw new IllegalArgumentException(
            "Email, first name and last name are required");
    }

    double marks;
    try {
        marks = Double.parseDouble(txtMarks.getText().trim());
    } catch (NumberFormatException ex) {
        throw new IllegalArgumentException(
            "Marks must be a number");
    }

    if (marks < 0 || marks > 10) {
        throw new IllegalArgumentException(
            "Marks must be between 0 and 10");
    }

    return new Student(email, first, last, marks);
}
```

6.15. Step 15 – CREATE handler

```
@FXML
private void handleCreate() {
    try {
        Student input = readForm();
        studentService.create(input);

        refreshTable();
        clearForm();
        showInfo("Student created");
    } catch (IllegalArgumentException |
             IllegalStateException ex) {
        showError(ex.getMessage());
    } catch (RuntimeException ex) {
        ex.printStackTrace();
        showError("Unable to create student");
    }
}
```

6.16. Step 16 – UPDATE handler

```
@FXML
private void handleUpdate() {
    if (selectedStudentId == null) {
        showError("Select a student first");
        return;
    }

    try {
        Student input = readForm();
        input.setId(selectedStudentId);

        studentService.update(input);

        refreshTable();
        clearForm();
        showInfo("Student updated");
    } catch (RuntimeException ex) {
        showError(ex.getMessage());
    }
}
```

PRODUCTION: Nếu entity ID setter bị hạn chế, Service có thể nhận UpdateStudentCommand/id + fields thay vì detached entity. Đây là production-safe direction.

6.17. Step 17 – DELETE handler

```
@FXML
private void handleDelete() {
    if (selectedStudentId == null) {
        showError("Select a student first");
        return;
    }

    if (!confirm("Delete selected student?")) {
        return;
    }

    try {
        studentService.delete(selectedStudentId);
        refreshTable();
```

```
        clearForm();
        showInfo("Student deleted");
    } catch (RuntimeException ex) {
        showError(ex.getMessage());
    }
}
```

6.18. Step 18 – Clear & Refresh handlers

```
@FXML
private void handleClear() {
    clearForm();
}

@FXML
private void handleRefresh() {
    refreshTable();
}

private void clearForm() {
    selectedStudentId = null;
    txtEmail.clear();
    txtFirstName.clear();
    txtLastName.clear();
    txtMarks.clear();
    tblStudents.getSelectionModel().clearSelection();
    txtEmail.requestFocus();
}
```

6.19. Step 19 – Alert helpers

```
private void showInfo(String message) {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setHeaderText(null);
    alert.setContentText(message);
    alert.showAndWait();
}

private void showError(String message) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setHeaderText("Operation failed");
    alert.setContentText(message);
    alert.showAndWait();
}

private boolean confirm(String message) {
    Alert alert = new Alert(
        Alert.AlertType.CONFIRMATION,
        message,
        ButtonType.YES,
        ButtonType.NO
    );
    return alert.showAndWait().orElse(ButtonType.NO)
        == ButtonType.YES;
}
```

6.20. Step 20 – Run end-to-end CRUD

```
mvn clean javafx:run
```

Action	UI expected	DB expected
Create	row appears + form clear	INSERT
Select	form populated	no DB change

Action	UI expected	DB expected
Update	row shows new values	UPDATE
Delete	row disappears	DELETE
Refresh	table matches DB	SELECT

6.21. Step 21 – Break-It: fx:id mismatch

Đổi `fx:id="txtEmail"` thành `txtEmailX` nhưng giữ Controller field `txtEmail`; chạy lại.

Expected: injection không đúng và Controller có thể NPE khi dùng field. Diagnose bằng FXML/Controller exact-name check.

BREAK-IT: Khôi phục sau experiment.

6.22. Step 22 – Break-It: remove `opens ... to javafx.fxml`

Trong modular project, xóa `opens fu.edu.sba301.ui.controller to javafx.fxml;` rồi chạy.

Expected: reflection/FXML access failure. Ghi nested exception và restore module directive.

6.23. Step 23 – Break-It: wrong event handler

Đổi `onAction="#handleCreate"` thành method không tồn tại.

Expected: FXMLLoader fails to resolve handler; inspect FXMLLoadException.

6.24. Step 24 – Break-It: invalid marks

Nhập `abc`, -1, 11.

Expected: UI validation message; Service/DB không được gọi mutation.

Evidence: breakpoint ở Service `create/update` không hit cho invalid form.

6.25. Step 25 – Duplicate email / service business error

Create Student với email đã tồn tại.

Expected: Controller nhận meaningful Service/application exception và hiển thị message; không show raw SQL exception.

6.26. Step 26 – UI thread awareness experiment

Tạm thêm `Thread.sleep(3000)` trong handler trước refresh để quan sát UI freeze. Sau đó xóa ngay.

BREAK-IT: Mục tiêu chỉ chứng minh event handler chạy trên JavaFX Application Thread. Không giữ sleep trong code.

6.27. Step 27 – Background Task production extension

```
Task<List<Student>> task = new Task<>() {
    @Override
    protected List<Student> call() {
        return studentService.getAll();
    }
};

task.setOnSucceeded(e ->
    data.setAll(task.getValue()));

task.setOnFailed(e ->
    showError("Unable to load students"));

Thread worker = new Thread(task);
worker.setDaemon(true);
worker.start();
```

PRODUCTION: Course không bắt buộc async cho CRUD nhỏ; đây là production direction khi DB call đủ lâu để ảnh hưởng responsiveness.

6.28. Step 28 – Controller integration test checklist

Case	Expected
FXML loads	No FXMLLoader exception
@FXML injection	all fields non-null after load
Initial refresh	Table shows service data

Case	Expected
Row selection	form + selected ID synchronized
Create valid	service called once + refresh
Create invalid	service not called
Update no selection	blocked
Delete cancel	service not called
Service error	error feedback shown

6.29. Step 29 – Integration Gate G1–G15

Gate	Evidence
G1 Build	Maven JavaFX project compile
G2 Module	controls/fxml + opens correct
G3 FXML	student-view loads
G4 Injection	@FXML fields non-null
G5 Service	Controller depends on interface
G6 Table	columns/items configured
G7 Selection	row → form
G8 Create	UI→Service→DB
G9 Update	selected ID + DB update
G10 Delete	confirmation + DB delete
G11 Validation	invalid input blocked
G12 Refresh	UI matches persistence
G13 Error	meaningful feedback
G14 Break/Fix	FXML/module error diagnosed
G15 Explain	FXML vs Controller vs Service responsibilities

6.30. Bài tập tổng hợp

1. Hoàn thiện Student-view.fxml bằng Scene Builder hoặc XML.
2. Thiết lập module-info đúng cho JavaFX/FXML.
3. Wire IStudentService vào Controller ngoài persistence layer.
4. Bind 5 TableColumns và ObservableList.
5. Implement selection-to-form.
6. Implement Create/Update/Delete/Clear/Refresh.
7. Implement validation marks/email/name.
8. Implement confirmation và alert helpers.
9. Chạy ba Break-It cases: fx:id, handler, module opens.
10. Chụp evidence UI + DB cho full CRUD.
11. Viết note giải thích vì sao Controller không chứa Hibernate Session.
12. Đề xuất background-task strategy cho DB operation chậm.

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ Slot 31
JavaFX	Java desktop UI toolkit	Student app
Application	JavaFX lifecycle base class	StudentApplication
Stage	Top-level window	primary stage
Scene	Container for scene graph	new Scene(root)
Node	Base visual object	Button/TextField
Scene Graph	Hierarchy of UI nodes	BorderPane→VBox→controls
FXML	XML UI markup	student-view.fxml
FXMLLoader	Loads FXML object graph	loader.load()
fx:controller	Controller class declaration	StudentController
fx:id	FXML object identifier	txtEmail
@FXML	Reflection-access annotation	private TextField
initialize()	Post-injection setup method	configure table
Controller	UI interaction coordinator	StudentController
TableView	Tabular control	tblStudents
TableColumn	Column definition	colEmail
ObservableList	Observable collection	data
SelectionModel	Selected-row model	getSelectionModel()
PropertyValueFactory	Property mapping helper	"email"
Event Handler	Method reacts to event	handleCreate
Alert	JavaFX dialog	error/info
JavaFX Application Thread	UI thread	event handlers
Task	Background operation abstraction	load students
module-info.java	JPMS module descriptor	requires/opens
opens	Runtime reflection access	controller→javafx.fxml

Thuật ngữ	Định nghĩa	Ví dụ Slot 31
Scene Builder	Visual FXML editor	student-view.fxml

VIII. RESEARCH BASE VÀ AI-BASED LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

8.1. Quy tắc AI

- Không paste DB credentials hoặc sensitive student data.
- AI-generated FXML phải compile/load và được Scene Builder/runtime kiểm.
- Claim về FXML lifecycle/module phải verify bằng OpenJFX official docs và debugger.
- AI không được đề xuất Controller mở Hibernate Session để 'đơn giản hóa'.
- Current ecosystem phải tách khỏi Course Baseline.

8.2. NHÓM I – Workflow / execution rule

PROMPT I.1: Trace FXMLLoader từ resource URL đến object graph, controller creation, field injection và initialize().

PROMPT I.2: Trace click Create từ Button event tới Service và DB, rồi refresh TableView.

PROMPT I.3: Trace row selection tới selectedStudentId và form population.

PROMPT I.4: Trace Update từ selected row đến Service; chỉ ra state nào là UI state vs entity state.

PROMPT I.5: Trace Delete confirmation tới Service/DB và TableView refresh.

PROMPT I.6: Trace FXMLLoadException do fx:id mismatch và xác định điểm lỗi.

PROMPT I.7: Trace module reflection error khi controller package không opens to javafx.fxml.

PROMPT I.8: Trace ObservableList setAll() làm TableView đổi dữ liệu.

PROMPT I.9: Trace UI freeze khi event handler thực hiện blocking DB call.

PROMPT I.10: Vẽ sequence diagram JavaFX Controller ↔ Service ↔ Repository ↔ DAO cho full CRUD.

8.3. NHÓM II – 10 prompt kỹ thuật nâng cao

PROMPT II.1: So sánh PropertyValueFactory với typed cellValueFactory/lambda và JavaFX Properties về refactor safety.

PROMPT II.2: Phân tích FXMLLoader controller lifecycle: constructor, @FXML injection, initialize, setter injection; đề xuất dependency-injection-safe pattern.

PROMPT II.3: Phân tích `fx:controller` vs `FXMLLoader.setController()` vs controllerFactory và các lỗi khi trộn chúng.

PROMPT II.4: Thiết kế validation architecture cho form Student: field-level, cross-field, service-level và database-level.

PROMPT II.5: Phân tích ObservableList mutation vs full reload sau CRUD về consistency/performance.

PROMPT II.6: Phân tích TableView selection state sau refresh và cách tránh update/delete nhầm row.

PROMPT II.7: Thiết kế async loading bằng Task/Service với cancellation, error, progress và FX-thread update.

PROMPT II.8: Phân tích memory leak risks từ listeners/controller navigation và cách remove listener/dispose.

PROMPT II.9: Phân tích JPMS exports vs opens cho FXML reflection và custom controls.

PROMPT II.10: Thiết kế controller tests dùng fake service + FXML integration tests; phân biệt unit/UI integration scope.

8.4. NHÓM III – 10 prompt production engineering

PROMPT III.1: Refactor StudentController thành presentation adapter mỏng với StudentFormModel/ViewModel; nêu lợi ích và trade-off.

PROMPT III.2: Thiết kế composition root/controllerFactory để inject Service vào nhiều FXML controllers mà không dùng static singleton.

PROMPT III.3: Thiết kế JavaFX navigation giữa Login và Student screen nhưng chỉ tạo một application-level persistence/service graph.

PROMPT III.4: Thiết kế background executor policy cho DB operations: queue size, cancellation, shutdown, UI feedback.

PROMPT III.5: Thiết kế error taxonomy từ PersistenceException → Service exception → user-facing JavaFX message.

PROMPT III.6: Thiết kế paging/search UI cho 100k Students thay vì load toàn bộ TableView.

PROMPT III.7: Thiết kế optimistic-lock conflict UX: record đã bị người khác sửa thì JavaFX nên hiển thị gì?

PROMPT III.8: Thiết kế accessibility/keyboard navigation cho Student form/TableView và validation feedback.

PROMPT III.9: Thiết kế packaging/deployment bằng Maven + jlink/jpackage; tách config/credentials khỏi image.

PROMPT III.10: Review JavaFX-Hibernate architecture và tìm 20 coupling/testability/threading/resource-management risks.

8.5. NHÓM IV – 10 prompt current ecosystem / community trends

PROMPT IV.1: Đọc OpenJFX official Getting Started hiện tại và kiểm Maven/Gradle/module setup cho JavaFX/FXML; ghi access date.

PROMPT IV.2: Đọc current OpenJFX FXML documentation và xác minh controller/@FXML/module reflective-access rules.

PROMPT IV.3: Khảo sát current JavaFX releases/LTS options nhưng không nâng course project; tạo Course Baseline vs Current Radar.

PROMPT IV.4: Khảo sát Scene Builder current workflow/tooling và compatibility với FXML version trong course.

PROMPT IV.5: Khảo sát jlink/jpackage current JavaFX desktop distribution flow và tạo deployment checklist.

PROMPT IV.6: Khảo sát TestFX/Monocle hoặc JavaFX testing ecosystem hiện tại; phân loại stable/useful/optional.

PROMPT IV.7: Khảo sát MVVM libraries/patterns cho JavaFX hiện tại và so sánh với FXML Controller pattern của course.

PROMPT IV.8: Khảo sát reactive bindings/properties ecosystem và khi nào đáng dùng thay manual form synchronization.

PROMPT IV.9: Khảo sát virtualized controls/paging strategies cho large TableView datasets.

PROMPT IV.10: Tạo '2026 JavaFX Desktop Radar': FXML, modules, Maven, Scene Builder, concurrency, testing, jlink/jpackage, MVVM, accessibility, observability.

8.6. AI Verification Worksheet

Claim	Nguồn	Experiment	Kết luận
FXML injection lifecycle	OpenJFX docs	breakpoint initialize	
opens required	OpenJFX docs/module test	remove opens	
Controller separation	source dependency graph	import audit	
UI thread blocking	JavaFX runtime	sleep experiment	
Current build setup	OpenJFX docs	mvn javafx:run	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

□ 1. Tôi giải thích Stage, Scene và scene graph.

□ 2. Tôi giải thích vai trò của FXML/FXMLLoader.

□ 3. Tôi biết fx:controller, fx:id và @FXML liên kết thế nào.

□ 4. Tôi biết initialize() chạy sau injection.

□ 5. Tôi cấu hình module-info cho javafx.controls/javafx.fxml.

□ 6. Tôi hiểu vì sao controller package cần opens khi dùng named module.

□ 7. Tôi bind được TableView/TableColumn/ObservableList.

□ 8. Tôi đồng bộ row selection với form.

□ 9. Tôi triển khai Create/Update/Delete qua Service.

□ 10. Tôi không truy cập DAO/Hibernate trực tiếp từ Controller.

□ 11. Tôi validate input và hiển thị lỗi có nghĩa.

□ 12. Tôi biết cách debug FXMLLoadException.

□ 13. Tôi chứng minh CRUD bằng UI + DB evidence.

□ 14. Tôi hiểu JavaFX Application Thread và UI freeze.

□ 15. Tôi biết production direction cho background tasks và dependency injection.

IX.1. Exit ticket

Câu hỏi	Trả lời
Vì sao không access @FXML field trong constructor?	
`fx:id` khác `fx:controller` thế nào?	
Controller nên gọi DAO hay Service?	
Nếu FXML controller package không `opens` thì lỗi gì có thể xảy ra?	
DB call dài trong event handler gây vấn đề gì?	

TÀI LIỆU THAM CHIẾU & SOURCE TRACEABILITY

- Course source: Chapter 18 pages 21–39 – JavaFX project, architecture, module-info, project reference, student-view.fxml/Scene Builder, StudentController, row selection, Student CRUD, run.
- Course source: Lab 07 Activity 03 – Student List Page: JavaFX project, architecture, module, reference project, FXML, StudentController, list/selection and CRUD.
- Official OpenJFX Getting Started – Maven/modular JavaFX setup; FXML projects require javafx.fxml.
- Official OpenJFX FXML documentation – FXML object graph, controller methods, @FXML and module reflective access.

NOTE: Current ecosystem note: OpenJFX official documentation continues to support Maven/Gradle modular workflows. For named modules using FXML, controller/custom-node packages referenced by FXMLLoader must be reflectively accessible, commonly via `opens ... to javafx.fxml`. These current notes supplement rather than replace the course Chapter 18 baseline.